

UNIVERSIDADE FEDERAL DO RIO GRANDE — FURG

Centro de Ciências Computacionais — C3

Sistemas para Internet 2 — Turma U — 1º Semestre de 2026

Prof. Dr. André Prisco Vargas

TRABALHO DO PRIMEIRO BIMESTRE

Web Proxy com Controle de Conteúdo

Olá pessoal.

Ao longo das primeiras semanas da disciplina, estudamos como a Web funciona: o protocolo HTTP, o modelo cliente-servidor, a estrutura das requisições e respostas, e os fundamentos da comunicação em rede. Chegou a hora de colocar esse conhecimento em prática de forma integrada.

Neste trabalho, que pode ser em dupla, vocês devem implementar um **web proxy**, que é um servidor intermediário que fica entre o navegador (ou qualquer cliente HTTP) e a Internet. Em vez de acessar um site diretamente, o cliente envia a requisição ao proxy, que deve decidir entre repassar o conteúdo, bloquear o acesso ou filtrar o conteúdo antes de entregá-lo.

Como devem saber, proxies são de fato usados em redes corporativas, bibliotecas, escolas e provedores de internet para controlar e registrar o tráfego. O que vocês vão construir é uma versão didática, claro, mas funcionalmente coerente com esse cenário.

O que o proxy deve fazer

O proxy recebe requisições no formato:

```
http://localhost:5000/http://www.exemplo.com
```

e executa uma das três ações a seguir, dependendo da URL solicitada:

Modo transparente — repassar o conteúdo

Se o site não estiver em nenhuma lista especial, o proxy faz a requisição ao servidor de origem, recebe o conteúdo e o devolve ao cliente **sem modificações**. O cliente deve receber exatamente o que receberia se acessasse o site diretamente.

Bloqueio de sites

Se o domínio estiver na **lista negra** (arquivo `blocked.json`), o proxy **não repassa o conteúdo**. Em vez disso, retorna ao cliente uma página HTML personalizada informando que o acesso foi bloqueado.

Essa página deve ser visualmente identificável — não pode ser apenas uma mensagem de erro genérica.

Exemplo de conteúdo do arquivo `blocked.json`:

```
{ "bloqueados": ["www.sitex.com", "redes-sociais.net", "joguinhos.io"] }
```

Filtro de conteúdo — substituição de palavras

Se o site não está bloqueado, mas o conteúdo HTML contém palavras cadastradas na **lista de palavras** (arquivo `words.json`), o proxy substitui essas palavras pelas alternativas configuradas **antes de entregar o conteúdo ao cliente**. O cliente recebe o HTML modificado sem perceber a substituição.

Exemplo de conteúdo do arquivo `words.json`:

```
{ "foda": "diabos", "merda": "macacos me mordam", "idiota": "ingênuo" }
```

A substituição deve ser `case-insensitive` — ou seja, *Merda*, *MERDA* e *merda* devem ser tratadas da mesma forma.

Log de acessos

Toda requisição recebida pelo proxy deve ser registrada em um arquivo de log (`log.json` ou `log.txt`), contendo no mínimo:

- Timestamp da requisição
- URL solicitada
- Ação executada: permitido, bloqueado ou filtrado

Tecnologias

A dupla é **livre para escolher** como implementar o servidor proxy. Flask e sockets TCP em Python são as tecnologias trabalhadas em aula, mas se tu já tiver experiência ou curiosidade em trabalhar com Node.js, Spring, PHP ou qualquer outra — podem utilizá-las.

A escolha deve ser **justificada no README do repositório**: por que a dupla escolheu essa tecnologia? Quais foram as vantagens e dificuldades percebidas em relação às alternativas?

Pode ser que eu não consiga ajudá-los com qualquer tecnologia, por isso garanto suporte técnico apenas para implementações em Python (Flask ou sockets). Duplas que escolherem outra tecnologia são responsáveis por resolver problemas conta própria 😊!

Componente	Tecnologia sugerida	Observação
Servidor proxy	Flask, sockets TCP ou outra tecnologia	Escolha da dupla — justificar no README
Requisições ao servidor de origem	requests (Python) ou equivalente	Biblioteca HTTP client na linguagem escolhida
Listas de configuração	Arquivos JSON	blocked.json e words.json
Log de acessos	Arquivo de texto (.txt ou .json)	Gerado automaticamente pelo proxy
Versionamento	Git + GitHub	Repositório da dupla, obrigatório

Entrega

A entrega é feita via GitHub com link apresentado no AVA. O repositório deve conter:

- Código-fonte completo do proxy
- Arquivos de configuração: `blocked.json` e `words.json`
- `README.md` com instruções claras de como instalar dependências, configurar as listas, executar o proxy e — se a dupla usou IA — como foi esse uso
- Arquivo de log gerado por ao menos uma sessão de testes

Repositório vazio ou inacessível na data de entrega equivale a não entrega.

Relatório técnico

Junto com o código, a dupla deve entregar um relatório técnico em PDF contendo:

- Justificativa da tecnologia escolhida
- Decisões de implementação relevantes — o que foi difícil, o que foi surpreendente
- Explicação de como o proxy lida com HTTPS: por que o filtro de conteúdo não funciona em sites HTTPS? O que seria necessário para funcionar?
- O que a dupla aprendeu sobre o protocolo HTTP ao implementar o proxy

A questão sobre HTTPS é obrigatória. Não é necessário implementar suporte a HTTPS — apenas explicar o fenômeno.

A ausência do relatório implica desconto de pontuação no total do trabalho, independentemente da qualidade do código.

Avaliação

O trabalho vale 6 pontos no bimestre, distribuídos conforme a tabela abaixo:

Critério	Descrição
Proxy básico (requisição e repasse)	O proxy recebe a URL, faz a requisição ao servidor de origem e retorna o conteúdo ao cliente corretamente.
Bloqueio de sites	Sites da lista negra retornam uma página de bloqueio personalizada, sem expor erros internos.
Filtro de palavras	O conteúdo HTML é modificado corretamente antes de ser entregue; pares de substituição são lidos do arquivo JSON.
Log de acessos	Cada acesso é registrado com timestamp, URL e ação tomada (permitido / bloqueado / filtrado).
Qualidade do código	Código legível, organizado em funções, sem duplicação desnecessária e com comentários nos trechos relevantes.
README e instruções	O README explica como instalar, configurar as listas e executar o proxy; qualquer pessoa consegue rodar seguindo as instruções.
Relatório técnico	Conforme já apresentei.

Desafio opcional

Para as duplas que queiram ganhar “estrelinhas”: implementem suporte ao método HTTP [CONNECT](#), permitindo que o proxy funcione como túnel para conexões HTTPS. O proxy não precisa inspecionar o conteúdo cifrado — apenas tunelizar a conexão TCP entre o cliente e o servidor de destino.

O desafio está aberto a todas as duplas e vale pontos extras na avaliação final da disciplina (a negociar).

Transparência no uso de IA

O uso de ferramentas de inteligência artificial — como Claude, ChatGPT, GitHub Copilot ou similares — é **opcional**. A dupla que optar por utilizá-las deve fazê-lo com um objetivo claro: **aprender mais e melhor**, não simplesmente obter uma nota. Vou considerar plágio o uso de IA para gerar código sem entendê-lo. O trabalho existe para que vocês aprendam, não para treinar Big Techs.

Caso a dupla utilize IA, deve incluir no relatório técnico uma **seção dedicada ao uso da ferramenta**, descrevendo: quais ferramentas foram utilizadas, como foram usadas (geração de código, esclarecimento de dúvidas, revisão, etc.), o que foi gerado, o que foi modificado e o que a dupla aprendeu com a interação. Esse relato deve ser honesto e detalhado.

A **apresentação presencial** vai usar esse relatório como base para as perguntas. Eu vou sentar ao lado da dupla, abrir o código e fazer perguntas sobre qualquer trecho — gerado por IA ou não. A dupla deve ser capaz de explicar cada decisão.

Por fim, já registro: este enunciado eu elaborei com auxílio de inteligência artificial. Conduzi as decisões de conteúdo e revisei as seções (escrevendo manualmente este trecho, por exemplo). No

entanto, sou responsável integral pelo resultado e conheço o caminho para a construção do trabalho. **Imagino que o trabalho de vocês vá seguir o mesmo processo.**

Bom Trabalho!!!

9. Referências e recursos

HTTP

- MDN Web Docs — HTTP: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP> — documentação completa em português sobre métodos, headers e status codes
- MDN — Proxies e tunelamento: https://developer.mozilla.org/en-US/docs/Web/HTTP/Proxy_servers_and_tunneling — explica proxy, túnel e o método CONNECT
- RFC 7230 — HTTP/1.1: <https://datatracker.ietf.org/doc/html/rfc7230> — especificação oficial do protocolo

Python

- Documentação do Flask: <https://flask.palletsprojects.com>
- Biblioteca requests: <https://requests.readthedocs.io> — cliente HTTP para buscar o conteúdo dos sites de origem
- Módulo socket (Python): <https://docs.python.org/3/library/socket.html> — para quem escolher a implementação com sockets TCP